

PYSPARK

Working with JSON Data

20 hands-on tasks — from schema parsing to nested aggregations

`from_json`

`StructType`

`explode`

`groupBy`

Agenda

Four parts, twenty tasks

1

Parsing JSON Basics

Flat fields, nested objects, and array-valued fields

4 tasks

2

Arrays & Explode

Turning JSON arrays into analyzable rows

4 tasks

3

Aggregations on Orders

Grouping, summing, sorting and averaging nested data

6 tasks

4

Filtering, Derived Columns & Data Quality

Renaming, sorting, counting, and handling nulls

6 tasks

PART 1

Parsing JSON Basics

Flat fields, nested objects, and array-valued fields

#1

#2

#3

#8

1

Parse flat JSON strings

PART 1 · PARSING JSON BASICS

Define a StructType schema and use `from_json()` to turn a raw JSON string column into typed, queryable columns.

```
schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("age", IntegerType())
])

parsed_df = json_df.withColumn(
    "data",
    from_json(col("json_string"), schema)
)

parsed_df.select("data.*").show(truncate=False)
```

KEY FUNCTIONS

■ StructType

■ from_json

■ "data.*"

Task 1 of 4 in this part

2

Parse nested JSON objects

PART 1 · PARSING JSON BASICS

Nest one `StructType` inside another to model a JSON object within an object, then drill down with dot notation.

```
address_schema = StructType([
    StructField("city", StringType()),
    StructField("country", StringType())
])
schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("address", address_schema)
])

parsed_df.select(
    "data.id", "data.name",
    "data.address.city",
    "data.address.country"
).show(truncate=False)
```

KEY FUNCTIONS

- Nested `StructType`
- dot notation

Task 2 of 4 in this part

3

Parse a JSON array field

PART 1 • PARSING JSON BASICS

Model a JSON array as `ArrayType(StringType())` to capture list-valued fields like a user's skills.

```
schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("skills",
        ArrayType(StringType()))
])

parsed_df.select("data.*") \
    .show(truncate=False)
```

KEY FUNCTIONS

■ `ArrayType`

Task 3 of 4 in this part

8

Build a full address string

PART 1 • PARSING JSON BASICS

Combine several parsed, nested fields into one readable string using `concat_ws()`.

```
address_schema = StructType([
    StructField("street", StringType()),
    StructField("city", StringType()),
    StructField("country", StringType())
])

parsed_df.select(
    "data.id", "data.name",
    concat_ws(", ",
        "data.address.street",
        "data.address.city",
        "data.address.country"
    ).alias("full_address")
).show(truncate=False)
```

KEY FUNCTIONS

concat_ws

alias

Task 4 of 4 in this part

PART 2

Arrays & Explode

Turning JSON arrays into analyzable rows

#4

#9

#10

#15

4

Explode an array of order structs

PART 2 · ARRAYS & EXPLODE

Parse an array of structs (each order), then use `explode()` so every order becomes its own row.

```
order_schema = StructType([
    StructField("order_id", IntegerType()),
    StructField("product", StringType()),
    StructField("quantity", IntegerType()),
    StructField("price", IntegerType())
])
schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("orders", ArrayType(order_schema))
])
orders_df = parsed_df.select(
    "data.id", "data.name",
    explode("data.orders").alias("order")
)
```

KEY FUNCTIONS

■ `ArrayType(StructType)`

■ `explode`

Task 1 of 4 in this part

9

Count elements in an array

PART 2 · ARRAYS & EXPLODE

Use `size()` to find how many items are inside a JSON array field, e.g. the number of skills per user.

```
parsed_df.select(  
    "data.id", "data.name",  
    "data.skills",  
    size("data.skills")  
        .alias("number_of_skills")  
) .show(truncate=False)
```

KEY FUNCTIONS

■ `size()`

Task 2 of 4 in this part

Check for a specific array value

PART 2 · ARRAYS & EXPLODE

Use `array_contains()` to filter rows whose array field includes a target value, e.g. users who know “Spark”.

```
parsed_df.select("data.*") \
    .filter(
        array_contains(
            col("skills"), "Spark"
        )
    ).show(truncate=False)
```

KEY FUNCTIONS

■ `array_contains`

■ `filter`

Task 3 of 4 in this part

Explode a simple string array

PART 2 · ARRAYS & EXPLODE

`explode()` also works on plain `ArrayType(StringType())` fields — not just arrays of structs — turning a product list into rows.

```
parsed_df.select(  
    "data.id", "data.name",  
    explode("data.products")  
        .alias("product")  
) .show(truncate=False)
```

KEY FUNCTIONS

■ `explode`

Task 4 of 4 in this part

PART 3

Aggregations on Orders

Grouping, summing, sorting and averaging nested data

#5

#11

#12

#13

#14

#20

5

Total spending per customer

PART 3 · AGGREGATIONS ON ORDERS

Group the exploded orders back by customer and sum quantity × price to get each customer's total spend.

```
orders_df.groupBy("id", "name").agg(  
  sum(  
    col("order.quantity") *  
    col("order.price")  
  ).alias("total_spending")  
) .show(truncate=False)
```

KEY FUNCTIONS

groupBy

agg

sum

Task 1 of 6 in this part

11

Line total for every order

PART 3 · AGGREGATIONS ON ORDERS

Compute quantity × price for each individual order row — a row-level calculation, no aggregation needed.

```
orders_df.select(  
    "id", "name",  
    "order.order_id", "order.product",  
    "order.quantity", "order.price",  
    (col("order.quantity") *  
     col("order.price")).alias("total")  
).show(truncate=False)
```

KEY FUNCTIONS

- column arithmetic

- alias

Task 2 of 6 in this part

Find the most expensive order

PART 3 · AGGREGATIONS ON ORDERS

Reuse the per-order total and sort descending with `orderBy()` to surface the highest-value order first.

```
orders_df.select(  
    "id", "name", "order.order_id",  
    "order.product", "order.quantity",  
    "order.price",  
    (col("order.quantity") *  
     col("order.price")).alias("total")  
) .orderBy(  
    col("total").desc()  
) .show(truncate=False)
```

KEY FUNCTIONS

■ `orderBy(desc)`

Task 3 of 6 in this part

13

Rank customers by total spending

PART 3 · AGGREGATIONS ON ORDERS

Combine `groupBy/agg` with `orderBy()` to list customers from highest to lowest total spend.

```
orders_df.groupBy("id", "name").agg(  
    sum(  
        col("order.quantity") *  
        col("order.price")  
    ).alias("total_spending")  
) .orderBy(  
    col("total_spending").desc()  
) .show(truncate=False)
```

KEY FUNCTIONS

- groupBy
- sum
- orderBy(desc)

Task 4 of 6 in this part

Find big spenders (> 1000)

PART 3 · AGGREGATIONS ON ORDERS

Filter after aggregation — the PySpark equivalent of SQL's HAVING clause — to keep only high-value customers.

```
orders_df.groupBy("id", "name").agg(  
    sum(  
        col("order.quantity") *  
        col("order.price")  
    ).alias("total_spending")  
) .filter(  
    col("total_spending") > 1000  
) .show(truncate=False)
```

KEY FUNCTIONS

- filter after agg (HAVING)

Task 5 of 6 in this part

Average order value per user

PART 3 · AGGREGATIONS ON ORDERS

Swap `sum()` for `avg()` in the same `groupBy/agg` pattern to get each customer's average order value.

```
orders_df.groupBy("id", "name").agg(  
    avg(  
        col("order.quantity") *  
        col("order.price")  
    ).alias("average_order_value")  
).show(truncate=False)
```

KEY FUNCTIONS

■ `avg()`

Task 6 of 6 in this part

PART 4

Filtering, Derived Columns & Data Quality

Renaming, sorting, counting, and handling nulls

#6

#7

#16

#17

#18

#19

6

Filter users by age

PART 4 · FILTERING, DERIVED COLUMNS & DATA QUALITY

Parse the JSON, then apply a plain `filter()` condition on a typed field — just like any other DataFrame column.

```
parsed_df.select("data.*") \
    .filter(col("data.age") > 25) \
    .show(truncate=False)
```

KEY FUNCTIONS

■ `filter`

Task 1 of 6 in this part

Derive a new column from JSON fields

PART 4 · FILTERING, DERIVED COLUMNS & DATA QUALITY

Once fields are extracted, use `withColumn()` to compute new columns from them, e.g. `price × quantity`.

```
parsed_df.select("data.*") \
    .withColumn(
        "total",
        col("price") * col("quantity")
    ).show(truncate=False)
```

KEY FUNCTIONS

■ `withColumn`

Task 2 of 6 in this part

Mark schema fields nullable=True so missing keys become nulls instead of parse errors, then fillna() to default them.

```
schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True)
])

parsed_df.select("data.*") \
    .fillna({"age": 0}) \
    .show(truncate=False)
```

KEY FUNCTIONS

■ nullable schema

■ fillna

Task 3 of 6 in this part

Rename extracted fields

PART 4 · FILTERING, DERIVED COLUMNS & DATA QUALITY

Use `alias()` on each selected column to give the parsed fields clearer, business-friendly names.

```
parsed_df.select(  
    col("data.id").alias("user_id"),  
    col("data.name").alias("user_name"),  
    col("data.age").alias("user_age")  
) .show(truncate=False)
```

KEY FUNCTIONS

■ `alias`

Task 4 of 6 in this part

Sort users by age

PART 4 · FILTERING, DERIVED COLUMNS & DATA QUALITY

`orderBy()` with `desc()` sorts the parsed DataFrame from oldest to youngest user.

```
parsed_df.select("data.*") \
    .orderBy(
        col("age").desc()
    ).show(truncate=False)
```

KEY FUNCTIONS

■ `orderBy(desc)`

Task 5 of 6 in this part

Count users by age

PART 4 · FILTERING, DERIVED COLUMNS & DATA QUALITY

`groupBy()` + `count()` summarizes how many users share each age value, sorted for readability.

```
parsed_df.select("data.*") \
    .groupBy("age") \
    .count() \
    .orderBy(col("age")) \
    .show()
```

KEY FUNCTIONS

■ `groupBy`

■ `count`

Task 6 of 6 in this part

Function Cheat Sheet

Every PySpark JSON function used across the 20 tasks

PARSE

- `from_json()`
- `StructType / StructField`
- `ArrayType()`

EXPLORE ARRAYS

- `explode()`
- `size()`
- `array_contains()`

SHAPE COLUMNS

- `select() + "data.*"`
- `withColumn()`
- `alias()`
- `concat_ws()`

AGGREGATE & SORT

- `groupBy() + agg()`
- `sum() / avg() / count()`
- `orderBy()`
- `filter()`

DATA QUALITY

- `nullable=True`
- `fillna()`